

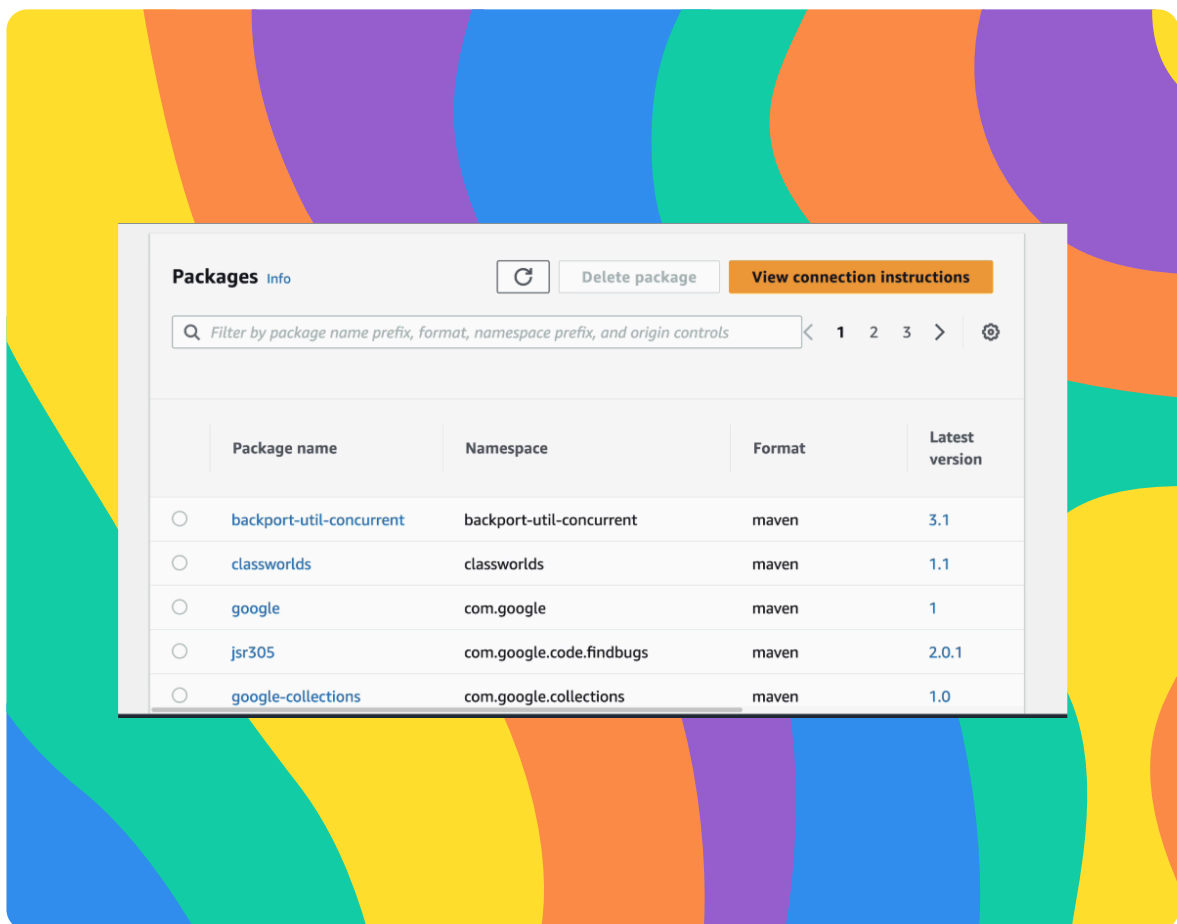


nextwork.org

Secure Packages with CodeArtifact



Dhruva Kashyap





Introducing Today's Project!

We are setting up AWS CodeArtifact to securely store and manage the project's dependencies. IAM roles and permissions are configured so the web app can access these packages safely. The connection is verified, and packages are uploaded to integrate CodeArtifact into the CI/CD pipeline.

Key tools and concepts

In this project, I learned how AWS CodeArtifact is used to securely store, manage, and distribute application dependencies and custom packages. I also gained hands-on experience with AWS Identity and Access Management (IAM) roles and policies to grant secure, temporary access to AWS services from an EC2 instance. Additionally, I understood how Maven integrates with CodeArtifact using configuration files and authorization tokens to enable controlled, reliable builds within a CI/CD pipeline.

Project reflection

This project took approximately 2 to 2.5 hours to complete. During this time, I set up dependency management using CodeArtifact, configured IAM roles and permissions, and verified the full package lifecycle. The hands-on steps helped reinforce both security and CI/CD concepts in a practical environment.



Dhruva Kashyap

NextWork Student

nextwork.org

I will work on the next project the following day as part of continuing the CI/CD learning series. This steady progression helps reinforce concepts by building on previous work. It also allows time to review and apply lessons learned before moving forward.

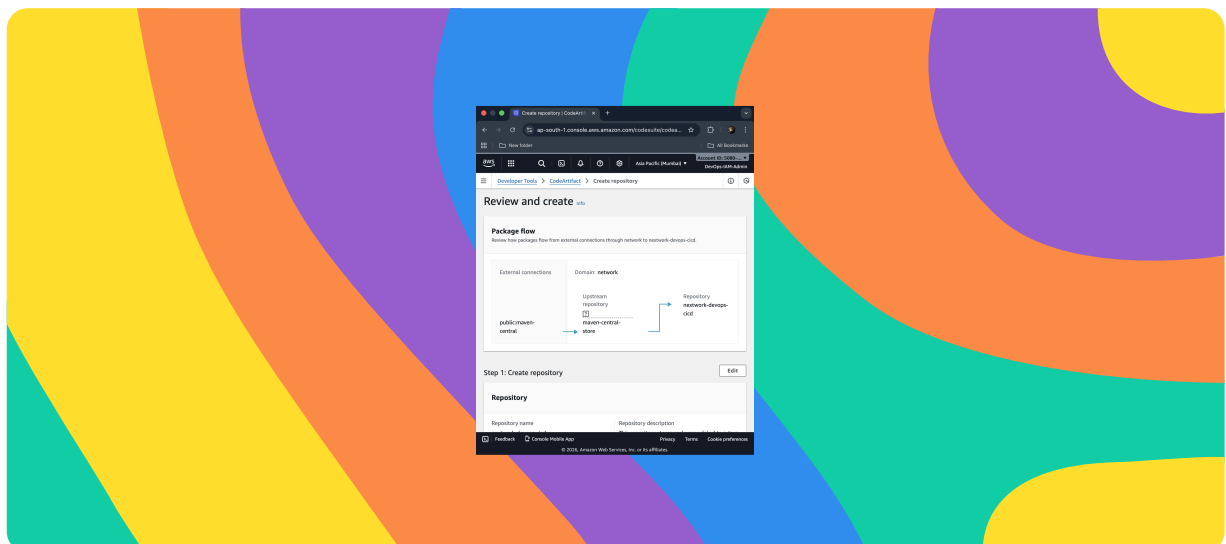


CodeArtifact Repository

AWS CodeArtifact is a managed artifact repository used to store and manage software packages and dependencies. It provides a secure and centralized place for teams to retrieve packages. This improves security, reliability, and control over dependency versions.

A domain in AWS CodeArtifact acts as a central container for multiple repositories. It allows permissions and security policies to be managed in one place. This ensures consistent access control across all repositories within the domain.

The upstream repository used is Maven Central Repository, configured as maven-central-store. It acts as a trusted public source for Java dependencies not found in the local repository. These packages are fetched once and cached securely in AWS CodeArtifact for future builds.





CodeArtifact Security

Issue

An authorization token is required to securely authenticate access to AWS CodeArtifact. It proves that the EC2 instance and tools like Maven have permission to access the repository. This prevents unauthorized users or services from downloading or uploading packages.

Resolution

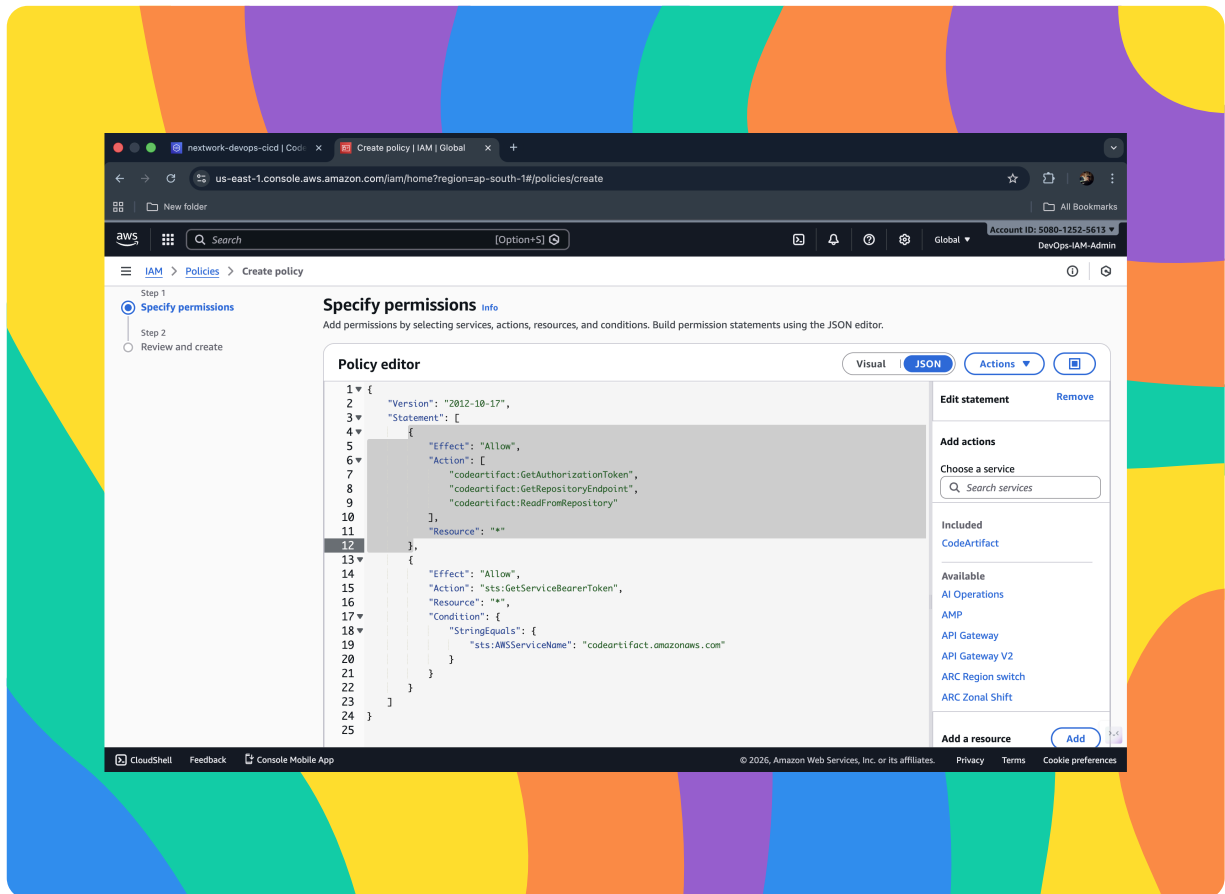
The permissions error was resolved by attaching the correct IAM role to the Amazon EC2 instance, which granted it access to AWS CodeArtifact. After allowing time for the role to propagate, the authorization token command was re-run successfully. This confirmed that the instance could now securely authenticate and access CodeArtifact using temporary credentials.

Using IAM roles is considered a security best practice because they provide temporary, automatically rotated credentials instead of long-term access keys. This allows services like Amazon EC2 to securely access other AWS services without storing sensitive credentials in code or configuration files. IAM roles also support the principle of least privilege, ensuring that resources have only the permissions they actually need.



The JSON policy attached to my role

This IAM policy, which is written in JSON, grants permission to authenticate with and read packages from AWS CodeArtifact. It allows fetching authorization tokens and repository endpoints so Maven can access dependencies. The STS permission enables temporary, secure access specifically for CodeArtifact operations, following least-privilege principles.



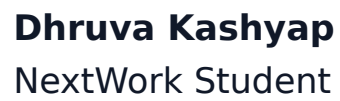


Maven and CodeArtifact

To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

The settings.xml file configures Maven with the repository URLs, authentication tokens, and rules required to connect to AWS CodeArtifact. It tells Maven where to fetch dependencies from and how to securely authenticate using the authorization token. This setup ensures that all dependency downloads are routed through CodeArtifact in a consistent and secure manner.

Compiling is the process of converting the project's source code into a format that the system can execute. It validates that the code and dependencies are correctly configured and work together as expected. This step ensures the application is ready to be built and run without errors.



The screenshot displays a Visual Studio Code editor window with a Docker Compose file named `settings.yml` open. The file is part of a project structure shown in the Explorer on the left, which includes a `network-web-project` directory. The Compose file defines a service `network-network-devops-cicd` with the following configuration:

```

1 settings:
2   |
3   -servers:
4     -server:
5       <id>network-network-devops-cicd/</id>
6       <username>aws/</username>
7       <password>${env.CODEARTIFACT_AUTH_TOKEN}-password/</password>
8     -server:
9       </server>
10    </servers>
11  </profile>
12  </profiles>
13  <id>network-network-devops-cicd/</id>
14  <activation>
15    <activeByDefault>true/</activeByDefault>
16  </activation>
17  <repositories>
18    <repository>
19      <id>network-network-devops-cicd/</id>
20      <url>https://network-588012525613.d.codeartifact.ap-south-1.amazonaws.com/aven/network-devops-cicd/</url>
21    </repository>
22  </repositories>
23  </profile>
24  </profiles>
25
26  <mirror>
27    <id>network-network-devops-cicd/</id>
28    <name>network-network-devops-cicd/</name>
29    <url>https://network-588012525613.d.codeartifact.ap-south-1.amazonaws.com/aven/network-devops-cicd/</url>
30    <mirrorOf>aws/mirrorOf/</mirrorOf>
31  </mirror>
32  </mirrors>
33  </settings>

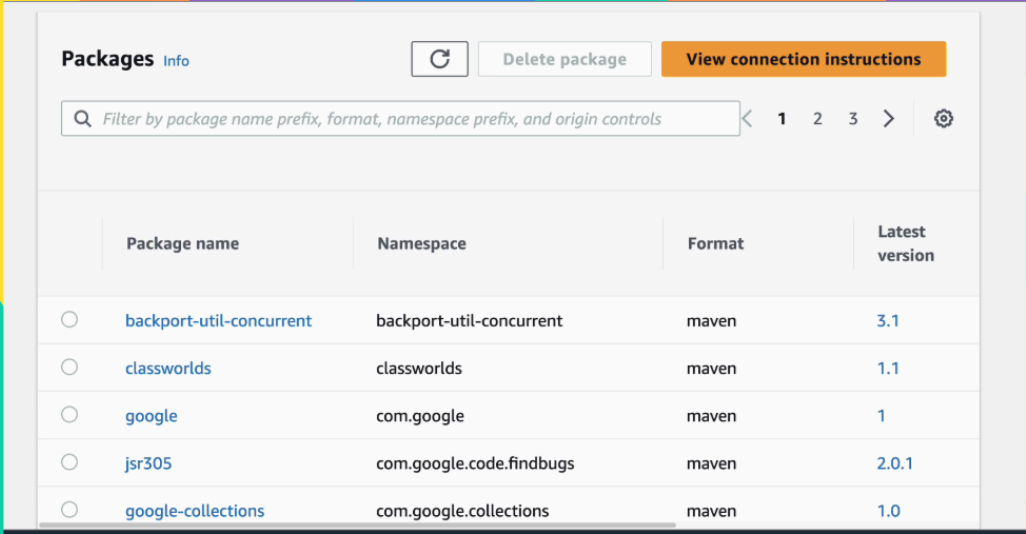
```

The IDE interface includes a file explorer on the left, a terminal at the bottom showing the command `SSH: ec2-66-1134-777.ap-south-1.compute.amazonaws.com`, and a status bar at the bottom indicating the current file is `Ln 2, Col 1`.



Verify Connection

After compiling the web app, the AWS CodeArtifact repository named nextwork-devops-cicd was checked in the console. The repository displayed a list of Maven packages that were automatically added during the build process. These packages confirmed that Maven successfully fetched dependencies via CodeArtifact and cached them from the upstream repository.



The screenshot shows the AWS CodeArtifact console interface. At the top, there's a 'Packages' header with an 'Info' link, a refresh button, a 'Delete package' button, and a 'View connection instructions' button. Below this is a search bar with the placeholder text 'Filter by package name prefix, format, namespace prefix, and origin controls'. To the right of the search bar are pagination controls showing '1 2 3' and a settings gear icon. The main part of the interface is a table with four columns: 'Package name', 'Namespace', 'Format', and 'Latest version'. The table lists five packages, each with a radio button in the first column.

	Package name	Namespace	Format	Latest version
☐	backport-util-concurrent	backport-util-concurrent	maven	3.1
☐	classworlds	classworlds	maven	1.1
☐	google	com.google	maven	1
☐	jsr305	com.google.code.findbugs	maven	2.0.1
☐	google-collections	com.google.collections	maven	1.0



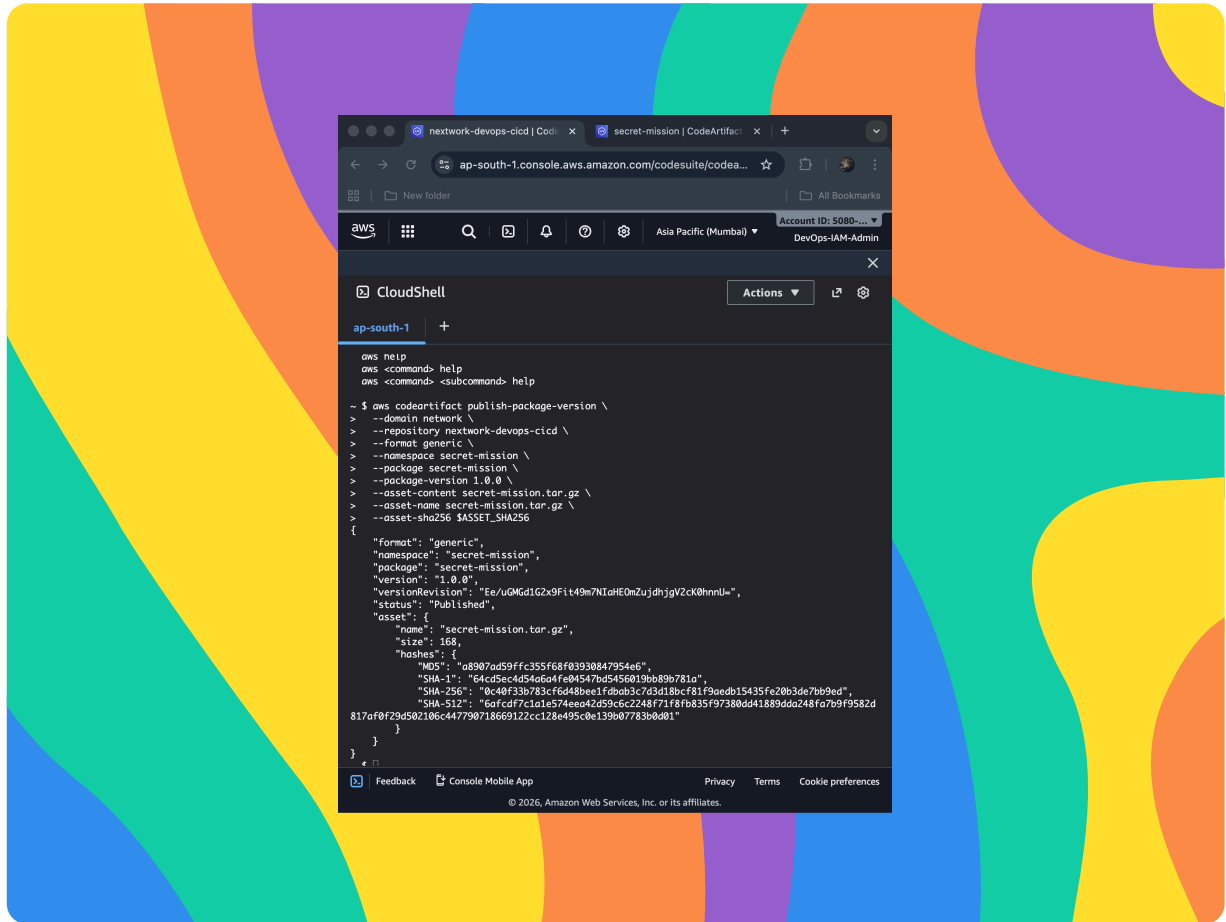
Uploading My Own Packages

This task focuses on creating and publishing a custom package to AWS CodeArtifact. It demonstrates how organizations manage and distribute internal libraries that are not meant for public repositories. By publishing and consuming a custom package, the complete lifecycle of enterprise package management is understood.

I generated my own package by first creating a simple text file containing a custom message using a shell command. That file was then bundled and compressed into a single archive using a tar.gz format to simulate a distributable package. Finally, a Security hash (SHA256) was generated for the archive to ensure its integrity before publishing it to the repository.

The AWS CodeArtifact console displays the published package name, version number, and namespace within the selected repository. It also shows metadata such as the publish time, package origin, and the repository where it is stored. Additionally, the package details include asset information like file name and SHA256 hash, confirming the integrity of the uploaded package.

I validated the package by downloading it from AWS CodeArtifact using the AWS CLI and confirming a successful retrieval response. The downloaded archive was extracted to verify that the contents matched the originally published file. Finally, reading the file contents confirmed that the package was intact and correctly managed through the publish-and-consume lifecycle.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

